Задача 3 (Резерфорд).

Доступ к информации на шаге вылета ядра гелия из золота.

```
~::UserSteppingAction(const G4Step* step)
{
  G4StepPoint* point1 = step->GetPreStepPoint();
  G4StepPoint* point2 = step->GetPostStepPoint();
……
  if (strcmp("G4_Au",step->GetPreStepPoint()->GetMaterial()->GetName())==0)
  {
   if (strcmp("alpha",step->GetTrack()->GetDynamicParticle()->GetDefinition()->GetParticleName())==0)
   {
    if (step->IsLastStepInVolume())
    {
      this->v1=point1->GetPosition();
      this->v2=point2->GetPosition();
    }
   }
  }
……
}
```

Задача 3 (Резерфорд).

Доступ к информации на шаге влёта ядра гелия в детектор из кремния.

```
~::UserSteppingAction(const G4Step* step)
{
 ……

 if (strcmp("G4_Si",step->GetPreStepPoint()->GetMaterial()->GetName())==0)
 {
   if (strcmp("alpha",step->GetTrack()->GetDynamicParticle()->GetDefinition()->GetParticleName())==0)
  {
   if (step->IsFirstStepInVolume())
   {
    vect1=point1->GetPosition();
    vect2=point2->GetPosition();
   }
  }
 }
}
```

Генераторы случайных чисел в Geant4

- Реализация моделирования Monte Carlo основывается на применении генераторов случайных чисел для имитации вероятностной природы физических процессов.
- Генератор случайных чисел – это «приблизительно» случайный генератор, использующий детерминированный алгоритм с предсказуемыми результатами.
- Существуют различные алгоритмов генерации чисел, отличающиеся скоростью работы и качеством генерации «случайности».
- Генераторы случайных чисел принято называть “engine” (движок).
- Каждый генератор случайных чисел в качестве начальных данных или начальной точки генерации использует seed (зерно): целое число или набор целых чисел.
- Для фиксированного зерна данный движок всегда генерирует одинаковую последовательность случайных чисел.

Генераторы случайных чисел в Geant4

- Также генератор случайных чисел характеризуется состоянием или статусом.
  - После остановки генерации последовательности из N случайных чисел движок находится в некотором состоянии, которое может быть сохранено (в файле).
  - При загрузке этого состояния в качестве начального при следующем запуске движка начнётся продолжение генерации последовательности случайных чисел, такой же, как и в случае без прерывания.

- Движок генерирует последовательность случайных чисел в интервале]0,1[.
  Другие распределения могут быть получены из равномерного с применением соответствующих преобразований.

- Geant4 обеспечивает возможности выбора движка, установки значения зерна, сохранения или перегрузки статуса.

Генераторы случайных чисел в Geant4

Контроль состояния движка необходим для:

1. **Начальной установки значений зерна/зёрен.**
   Запуск 10^7 событий на многопроцессорном компьютере можно разбить на 10 статистически независимых запусков для 10^6 событий с различными значениями начальных зёрен.
   В результате обеспечивается ускорение расчёта в 10 раз.

2. **Сохранение статуса движка.**
   При аварийном завершении программы необходимо определить причину сбоя.
   Если сбой произошёл при моделировании 1000000-ого события нет необходимости моделировать предыдущие 999999 при повторном запуске.
   При запуске программы с использованием состояния движка для 999999-го события можно сразу запустить моделирование сбойного события и детально исследовать его.
   Сохранение состояния движка для каждого события требует значительных временных ресурсов, данную процедуру **НЕ СЛЕДУЕТ** использовать всегда или по умолчанию.

Выбор движка определяется соотношением между скоростью расчёта и необходимым качеством случайности при генерации ряда случайных чисел.

Генераторы случайных чисел в Geant4

Класс HepRandom – модуль CLHEP (Class Library for High Energy Physics)

Генератор случайных чисел – "engine"

Класс HepRandom - абстрактный класс с интерфейсом для каждого генератора случайных чисел и различных типов распределений случайных чисел.

CLHEP::HepRandom Class Reference

Public Member Functions

	HepRandom ()
	HepRandom (long seed)
	HepRandom (**HepRandomEngine** &algorithm)
	HepRandom (**HepRandomEngine** *algorithm)
virtual	**~HepRandom** ()
double	**flat** ()
void	**flatArray** (const **int** size, double *vect)
double	**flat** (**HepRandomEngine** *theNewEngine)
void	**flatArray** (**HepRandomEngine** *theNewEngine, const **int** size, double *vect)
virtual double	**operator()** ()
virtual std::string	**name** () const
virtual **HepRandomEngine** &	**engine** ()
virtual std::ostream &	**put** (std::ostream &os) const
virtual std::istream &	**get** (std::istream &is)

Static Public Member Functions

static **void**	**setTheSeed** (long seed, **int** lux=3)
static long	**getTheSeed** ()
static **void**	**setTheSeeds** (const long *seeds, **int** aux=-1)
static const long *	**getTheSeeds** ()
static **void**	**getTheTableSeeds** (long *seeds, **int** index)
static **HepRandom** *	**getTheGenerator** ()
static **void**	**setTheEngine** (**HepRandomEngine** *theNewEngine)
static **HepRandomEngine** *	**getTheEngine** ()
static **void**	**saveEngineStatus** (const char filename[]="Config.conf")
static **void**	**restoreEngineStatus** (const char filename[]="Config.conf")
static std::ostream &	**saveFullState** (std::ostream &os)
static std::istream &	**restoreFullState** (std::istream &is)
static std::ostream &	**saveDistState** (std::ostream &os)
static std::istream &	**restoreDistState** (std::istream &is)
static std::ostream &	**saveStaticRandomStates** (std::ostream &os)
static std::istream &	**restoreStaticRandomStates** (std::istream &is)
static **void**	**showEngineStatus** ()
static **int**	**createInstance** ()
static std::string	**distributionName** ()

Генераторы случайных чисел в Geant4

Класс HepRandomEngine - абстрактный класс с интерфейсом для каждого генератора случайных чисел.

Встроенные генераторы случайных чисел:

- HepJamesRandom
  (используется по умолчанию)

- DRand48Engine
  (используются системные функции стандартных библиотек C)

- MixMaxRng
  (генератор по умолчанию при генерации распределений)

- RanluxEngine
  (используется генератор языка FORTRAN77, задаётся уровень качества генерации в диапазоне от 1 до5, по умолчанию 3)

- RanecuEngine
  (используется генератор языка FORTRAN77, начальные зёрна задаются из таблицы по индексу)

CLHEP::HepRandomEngine Class Reference

Public Member Functions

	HepRandomEngine ()
virtual	**~HepRandomEngine** ()
bool	**operator==** (const **HepRandomEngine** &engine)
bool	**operator!=** (const **HepRandomEngine** &engine)
virtual double	**flat** ()=0
virtual **void**	**flatArray** (const **int** size, double *vect)=0
virtual **void**	**setSeed** (long seed, **int**)=0
virtual **void**	**setSeeds** (const long *seeds, **int**)=0
virtual **void**	**saveStatus** (const char filename[]="Config.conf") const =0
virtual **void**	**restoreStatus** (const char filename[]="Config.conf")=0
virtual **void**	**showStatus** () const =0
virtual std::string	**name** () const =0
virtual std::ostream &	**put** (std::ostream &os) const
virtual std::istream &	**get** (std::istream &is)
virtual std::istream &	**getState** (std::istream &is)
virtual std::vector< unsigned long >	**put** () const
virtual bool	**get** (const std::vector< unsigned long > &v)
virtual bool	**getState** (const std::vector< unsigned long > &v)
long	**getSeed** () const
const long *	**getSeeds** () const
virtual	**operator double** ()
virtual	**operator float** ()
virtual	**operator unsigned int** ()

Генераторы случайных чисел в Geant4

Проект Rand

Loader.cc

```
CLHEP::HepRandom::setTheEngine(new CLHEP::RanecuEngine);
CLHEP::HepRandom::setTheSeed(time(NULL)+getpid());
G4Random::showEngineStatus();
(*ofs_sn) << "seed=" << CLHEP::HepRandom::getTheSeed() <<G4endl;
(*ofs_sn) << "seeds=" << *CLHEP::HepRandom::getTheSeeds() <<G4endl;
```

Задание генератора

Задание начального зерна

Вывод статуса генератора

Запись начальных зёрен в info.txt

Уровень загрузки проекта

Сохранение состояния движка в файл Config.conf, загрузка состояния движка из файла Config.conf

```
CLHEP::HepRandom::saveEngineStatus();
CLHEP::HepRandom::restoreEngineStatus();
```

Сохранение состояния движка в файл с динамически генерируемым именем “Loader_*идентификатор процесса*.rndm”, загрузка состояния движка из файла “Loader_*идентификатор процесса*.rndm”.

```
std::ostringstream os2;
os2<<"Loader_"<<G4Threading::G4GetThreadId()<<".rndm";
G4Random::saveEngineStatus(os2.str().c_str());
G4Random::restoreEngineStatus(os2.str().c_str());
```

преобразование строки в C строку с нулевым символом в конце

Генераторы случайных чисел в Geant4

Проект Rand

PrimaryPart.cc

Уровень генерации вершины

Сохранение состояния движка в файл ./random/run0evt0.rndm, загрузка состояния движка из файла ./random/run0evt0.rndm

```
std::string fileName="./random/run0evt0.rndm";
G4Random::saveEngineStatus(fileName.c_str());
G4Random::restoreEngineStatus(fileName.c_str());
```

Задание начальной энергии частицы, распределённой по Гаусу со средним значением 500 МэВ и среднеквадратичным отклонением 100 МэВ

```
G4double EnergyProt =G4RandGauss::shoot(500.*MeV,100.*MeV);
(*f_prim) << "Initial energy="<<EnergyProt<<G4endl;
GProton->SetParticleEnergy(EnergyProt);
```



Вывод в файл info.txt

Генераторы случайных чисел в Geant4

Классы распределений в HepRandom:

- RandFlat

```
double fnum = RandFlat::shoot();          // fnum  ]0,1[
double fnum = RandFlat::shoot(n);         // fnum  ]0,n[
double fnum = RandFlat::shoot(-m,n);      // fnum  ]-m,n[
  long inum = RandFlat::shootInt(k);        // inum  [0,k[
  long inum = RandFlat::shootInt(-h,k);     // inum  [-h,k[
      int i = RandFlat::shootBit();      // it returns just a bit (0 or 1)
const int size=n;
double vect[size];
RandFlat::shootArray(size,vect);  // to fill an array "vect" of n
                                  // double flat values
```

- RandExponential

```
double m;
double num = RandExponential::shoot();   // (mean=1)
double num = RandExponential::shoot(m);  // (mean=m)
```

$$f(x;\lambda) = \begin{cases} \lambda e^{-\lambda x} & x \geq 0, \\ 0 & x < 0. \end{cases}$$

- RandGauss

```
double num = RandGauss::shoot();     // (mean=0) (stDev=1)
double num = RandGauss::shoot(m,s);  // (mean=m, stDev=s)
```

$$f(x) = \frac{1}{\sigma\sqrt{2\pi}} e^{-\frac{1}{2}\left(\frac{x-\mu}{\sigma}\right)^2}$$

- RandBreitWigner

```
double num = RandBreitWigner::shoot(m,g);   // (mean=m, gamma=g)
double num = RandBreitWigner::shoot(m,g,c); // (mean=m, gamma=g, cut=c)
```

$$f(E) = \frac{k}{(E^2 - M^2)^2 + M^2\Gamma^2}$$

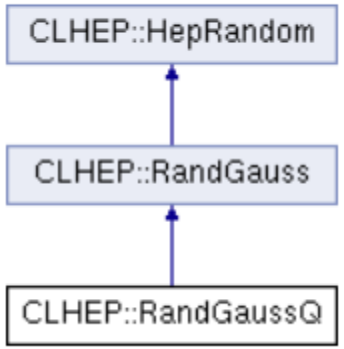
- RandPoisson

```
double m;
long num = RandPoisson::shoot(m);  // (mean=m)
```

$$P(X = k) = \frac{e^{-\lambda}\lambda^k}{k!}$$

Генераторы случайных чисел в Geant4

Randomize.hh File Reference



Среднее 0, среднеквадратическое отклонение 1.

```
#include <CLHEP/Random/Randomize.h>
#include <CLHEP/Random/RandFlat.h>
#include <CLHEP/Random/RandBit.h>
#include <CLHEP/Random/RandGamma.h>
#include <CLHEP/Random/RandGaussQ.h>
#include <CLHEP/Random/RandPoissonQ.h>
#include <CLHEP/Random/RandExponential.h>
#include <CLHEP/Random/RandGeneral.h>
```

```
#define  randomize_h  1
#define  G4RandStat  CLHEP::HepStat
#define  G4RandFlat  CLHEP::RandFlat
#define  G4RandBit  CLHEP::RandBit
#define  G4RandGamma  CLHEP::RandGamma
#define  G4RandGauss  CLHEP::RandGaussQ
#define  G4RandExponential  CLHEP::RandExponential
#define  G4RandGeneral  CLHEP::RandGeneral
#define  G4Random  CLHEP::HepRandom
#define  G4UniformRand()  CLHEP::HepRandom::getTheEngine()->flat()
```

G4double EnergyProt =G4RandGauss::shoot(500.*MeV,100.*MeV);